ACTIVITY – 11

IDA* Search Algorithm

Submitted by

D. Jeshurun Silas

Registration Number: 192525324

Subject: Artificial Intelligence
Activity: Write and explain the pseudocode for IDA* Search

1. Aim of the Activity

To understand the Iterative Deepening A* (IDA*) search algorithm and write its pseudocode. The algorithm is used to find a path from a starting point to a goal while using less memory than many traditional best-first search methods.

2. Introduction

Search algorithms help a computer find a route or solution by exploring different choices. For example, a computer may need to find a path through a map, solve a puzzle, or move a character through a game. IDA* is one search method designed for these kinds of problems.

IDA* means Iterative Deepening A-Star. It combines the depth-first style of exploring one path at a time with the cost estimate used by A* search. It repeats the search with a gradually increasing cost limit.

3. Important Terms

- **Start node:** The point where the search begins.
- **Goal node:** The target point the algorithm is trying to reach.
- **Path:** A sequence of connected nodes from the start to the goal.
- **g(n):** The actual cost already spent to reach the current node n.
- **h(n):** A heuristic: an estimate of the remaining cost from n to the goal.
- **f(n) = g(n) + h(n):** The estimated total cost of a solution path through node n.
- **Threshold:** The maximum f-value allowed during one search iteration.
- **Depth-first search:** A method that follows one path as far as it can before returning to try another.

4. Pseudocode for IDA* Search

The following pseudocode shows the main process. SEARCH returns FOUND when the goal is reached. If a node exceeds the current threshold, it returns that node's f-value so the next iteration can choose the smallest value that exceeded the limit.

```
IDA_STAR(start, goal)
    threshold = h(start)

    LOOP
        result = SEARCH(start, 0, threshold, goal)

        IF result == FOUND
            RETURN solution

        IF result == INFINITY
            RETURN FAILURE

        threshold = result


SEARCH(node, g, threshold, goal)
    f = g + h(node)
```

```
IF f > threshold
    RETURN f

IF node == goal
    RETURN FOUND

minimum = INFINITY

FOR each successor of node
    result = SEARCH(successor,
                    g + COST(node, successor),
                    threshold, goal)

    IF result == FOUND
        RETURN FOUND

    IF result < minimum
        minimum = result

RETURN minimum
```

5. Explanation of the Pseudocode

1. **Step 1 — Set the first threshold.** The algorithm begins by setting the threshold to h(start), the estimated distance from the start to the goal.
2. **Step 2 — Start a depth-first search.** SEARCH begins at the start node with g = 0 because no cost has been spent yet.
3. **Step 3 — Calculate f.** For every node, calculate f = g + h. This combines the cost already spent with the estimated cost still to come.
4. **Step 4 — Check the threshold.** If f is greater than the threshold, the algorithm does not explore that node in this round. It returns the f-value instead.
5. **Step 5 — Check the goal.** If the current node is the goal, the search has succeeded and the solution path is returned.
6. **Step 6 — Explore successors.** If the goal has not been reached, explore each connected next node. The path cost g increases by the cost of the road or move.
7. **Step 7 — Remember the next limit.** The algorithm remembers the smallest f-value that exceeded the current threshold. This becomes the threshold for the next round.
8. **Step 8 — Repeat.** The process repeats until a goal is found or no further threshold can lead to a solution.

6. Simple Example

Imagine searching for a destination on a map. The start is S and the destination is G. The algorithm first uses the heuristic estimate from S as its threshold. It explores paths whose estimated total cost f = g + h is within that limit. If the goal is not found, it raises the threshold to the smallest f-value that was too large in the previous round and searches again.

Term	Meaning in the example

g	Cost already travelled from S
h	Estimated cost remaining to G
f = g + h	Estimated total route cost
Threshold	The limit for exploring a path in the current round

7. Advantages of IDA*

- Uses much less memory than A* in many search problems because it explores paths depth-first.
- Uses a heuristic to guide the search toward promising routes.
- Can find an optimal solution when step costs and heuristic values meet the usual IDA* conditions, including an admissible heuristic.
- Useful for large search spaces, such as puzzles and route-finding problems.

8. Limitations of IDA*

- It may revisit the same nodes many times during different iterations.
- A poor heuristic can make the search slow.
- The pseudocode assumes a finite branching structure and that costs are handled appropriately.
- For graph problems with cycles, a practical implementation often needs path-cycle checking to avoid looping along the current path.

9. Applications

- Finding paths in maps and navigation problems.
- Solving puzzles such as sliding-tile puzzles.
- Searching game states and planning possible moves.
- Exploring large state spaces where memory is limited.

10. Result

The pseudocode for the IDA* Search Algorithm was written and explained. The activity demonstrates how the algorithm combines depth-first exploration with an f-cost threshold that increases when necessary.

11. Conclusion

IDA* is a heuristic search algorithm that repeatedly performs a depth-first search under a cost threshold. It calculates f(n) = g(n) + h(n), avoids exploring nodes that exceed the current threshold, and then raises the threshold to the smallest exceeded f-value. This approach is memory-efficient and can find an optimal path when the required heuristic and cost conditions are satisfied.